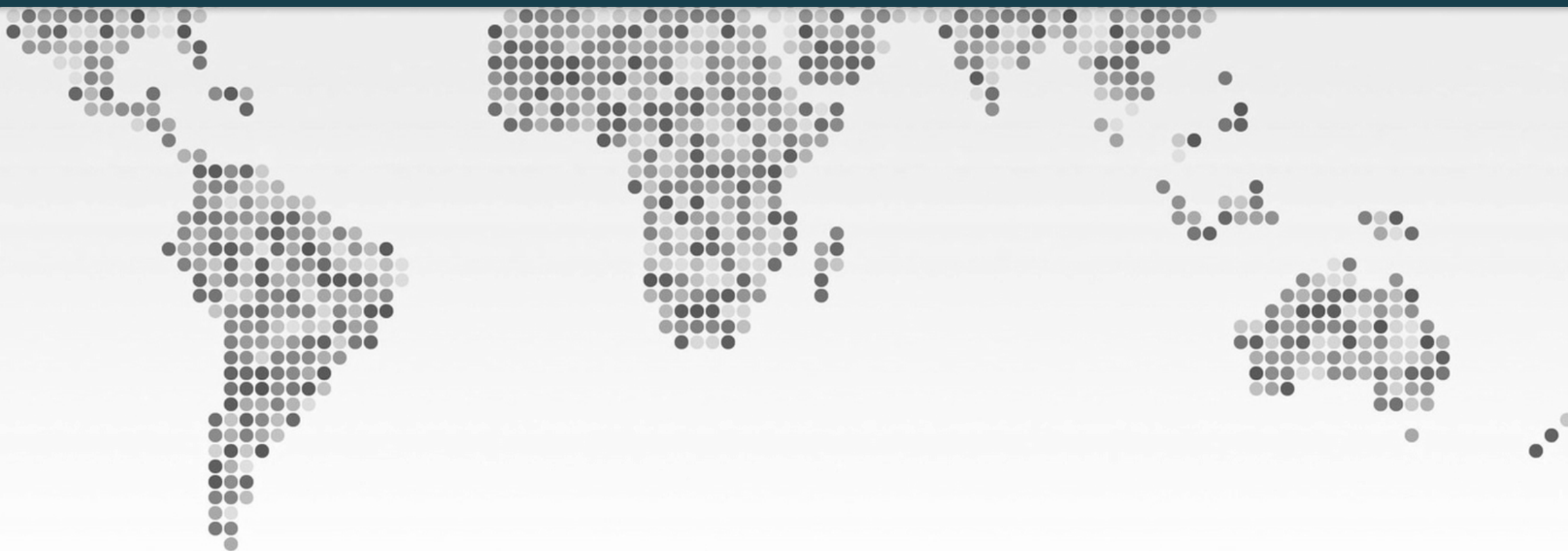


Software Engineering for Artificial Intelligence

Endterm In-Vivo Quiz



Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: Technical Debt and Bad Smells

Q1. Define the concept of technical debt, focusing on the debt called Feedback Loop and its impact on non-functional properties (including quality assurance, fairness, and security)

Model complexity

- Entanglement;
- Correction cascades;
- Undeclared consumers.

Data dependencies

- Unstable data dependency;
- Underutilized data dependencies.

Feedback loops

- Direct feedback loop;
- Hidden feedback loop.

ML systems anti-patterns

- Glue code;
- Pipeline jungle;
- Dead experimental codepaths;
- Abstraction debt;
- Plain-old-data type;
- Multiple language;
- Prototype.

One of the key features of live machine learning systems is that they often end up influencing their own behavior if they update over time, making hard to isolate their individual properties.

Problem: This is what we call *feedback loop*, a form of analysis debt where you cannot easily assess the behavior of a model before releasing it.

Let's consider an example: You have two smart systems operating on a web application: the first selects products to show, the second selects the related reviews.

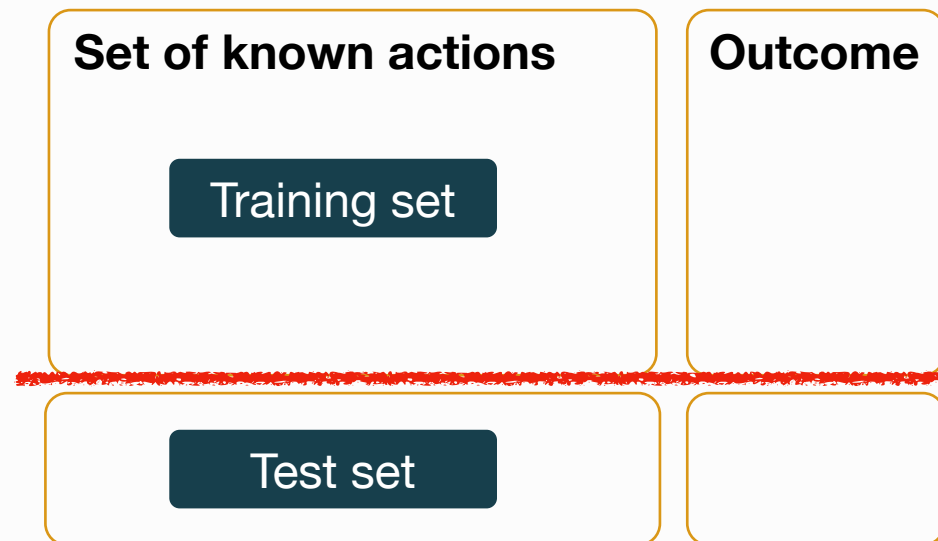
Improving one system may lead to changes in behavior in the other, as users begin clicking more or less on the other components in reaction to the changes.

This is a case of *hidden* feedback loop: a solution is still unknown, even if this problem may possibly lead to catastrophic consequences. For instance, consider two stock-market prediction models from two different investment companies. Improvements (or, more scarily, bugs) in one may influence the bidding and buying behavior of the other.

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: Validation

Q2. Describe the differences between the percentage-split and k-fold cross validation strategies, focusing on their potential limitations.



Well, this is not really a low-intensive-brain-activity method. The choice of the percentage may have a strong influence the resulting performance —> if you train with less data, you lose prediction capabilities, but if you train with more data you lose in terms reliability of the evaluation!

As some of you might have heard already, the 70%-30% rule (70% training, 30% test) seems to be the most established one.

My empirical soul must say something here: there is **NO empirical study** showing that this is a good way to configure the validation procedure. In other terms, this rule emerged anecdotally, which is quite ironic: an empirical method to validate machine learning system has never been subject of empirical validation...

This sounds weird, and it is weird indeed. However, it comes with two main questions: (1) What is the right splitting percentage to use? (2) Who ensures that my validation is reasonable enough to consider it good?

Who knows?

No one!

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: Validation

Q2. Describe the differences between the percentage-split and k-fold cross validation strategies, focusing on their potential limitations.

Set of known actions	Outcome

In cross-validation you split the dataset in n parts, called *folders*, and you validate the model using $n-1$ folders as training and the remaining folder as test. The process is then repeated so that each folder is used as test exactly once.

Typically, a 10-fold cross-validation is applied, with each folder defined randomly.

Sad but true, even the idea of using $n=10$ has never been empirically assessed. Nonetheless, it is way more credible because you experiment the model with (slightly) different training data and (slightly) different test data.

Cross-fold validation is the standard *de-facto* in model validation. However, someone might ask: when validating a model using this strategy, where are we going to preprocess data and configure hyper-parameters?

Do you remember the F1 driver parallel? Preprocessing and configuration are required to let the machine learner have the best possible environment to make predictions: yet, such a best possible environment should be assessed against a real case scenario which might diverge from the ideal case.

Hence, the test sets might be as real as possible!

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: Validation

Q3. Describe how the preprocessing and configuration procedures should be pursued when applying a k-fold cross validation.

Set of known actions	Outcome

Each fold used as test set should be left untouched.

At each iteration, the training data must be subject of the preprocessing and configuration steps.

This is an additional reason why experimenting with machine learning systems takes a lot: in the validation phase, you need to run expensive computation multiple times!

And the story is not yet at the end. While this is the *de-facto* standard for model validation, cross-validation still suffers from a key problem.

Quality Assurance of Artificial Intelligence Solutions: Validation

We need something else. As some of you might have heard already, cross-validation can come to rescue.

Set of known actions	Outcome

In cross-validation you split the dataset in n parts, called *folders*, and you validate the model using $n-1$ folders as training and the remaining folder as test. The process is then repeated so that each folder is used as test exactly once.

Typically, a 10-fold cross-validation is applied, with each folder defined randomly.

The initial random split of the folders might impact the evaluation procedure —> we might be (un)lucky with the split, biasing the results.

Dealing with randomness: just re-run the validation multiple times (e.g., 10 times) and analyze the results using statistics!

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: Validation

Q4. Describe the main problems of k-fold validation, other than the corresponding mitigation strategies.

Problem #1: Suppose you have a highly unbalanced problem. Independently from the re-running way of operating, you may still have issues when training models because of the lack of data —> in machine learning systems, if you do not have enough examples, you cannot learn!

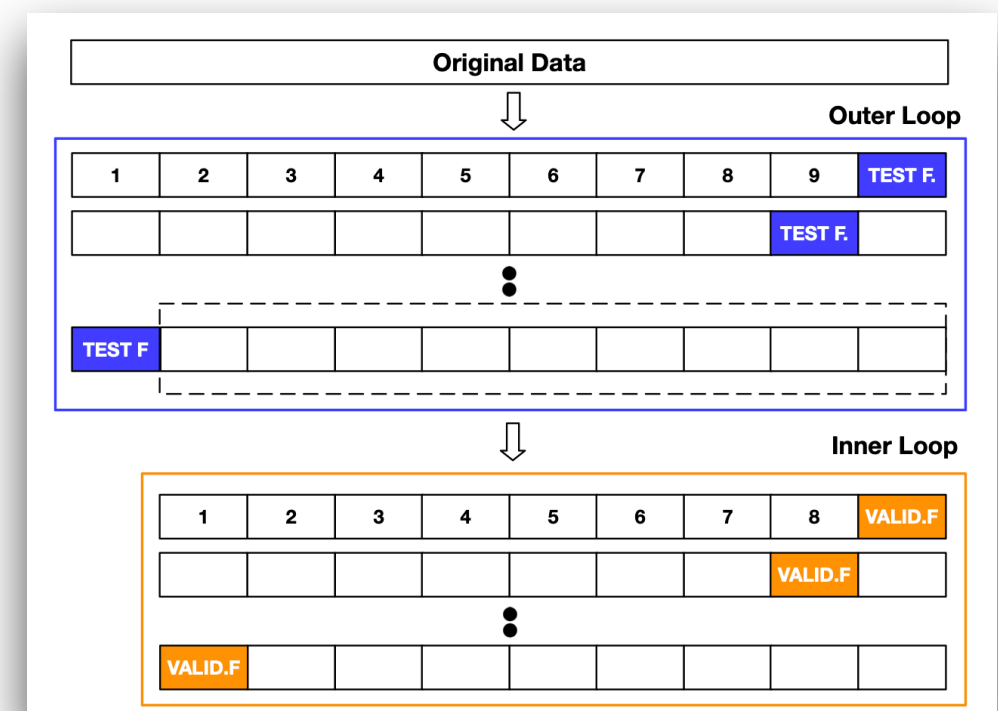
Solution: Perform a pseudo-random splitting of the folders. Rather than relying on randomness completely, you can apply a stratified sample approach where the examples of the minority class are equally distributed among the folders. In this way, you can still rely on seemingly real datasets!

Problem #2: To configure hyper-parameters of the learners, you might want to rely on an independent set to increase the degree of reality.

Solution: Cross-fold nested cross validation.

Split the entire set of data in n folds. One of these folds is retained as test and left untouched until the end of the computations done in the inner loop.

The remaining $n-1$ folds are instead used within the inner loop: one fold is reserved for the validation, while $n-2$ of them are used to train. You use the validation set to experiment with parameters, using an inner cross-validation.



Software Engineering for Artificial Intelligence: Quality Assurance

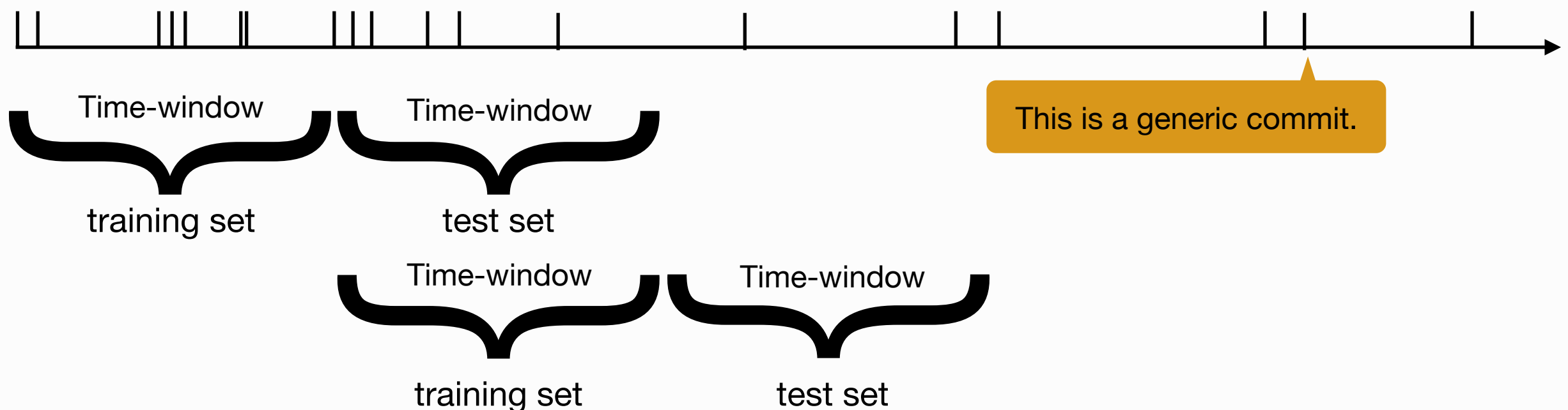
Quality Assurance of Artificial Intelligence Solutions: Validation

Q4. Describe the main problems of k-fold validation, other than the corresponding mitigation strategies.

Problem #3: Suppose you have temporal data, namely you want to do predictions based on data that follow a temporal distribution.

Let's do a software engineering example. You want to use the previous commit data to predict the likelihood of a new change introducing a security vulnerability.

You have a bunch of commits to learn from, and you want to use them.



Hence, time-sensitive validation. You iteratively train on data of a specific time window and test on the subsequent time window. In this way you preserve temporal data and experiment with a more realistic use case. Yet, you have to tune the time window: this is strictly dependent on the problem.

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: Testing

Q5. What is metamorphic testing and how can it be applied for the problem of input testing?

Input testing

Input level tests analyze the training data used to train the ML components as well as the input data used at prediction time.

Based on this example, let's define the key terms more formally.

Morph/Transform. An input modification/mutation created ad-hoc and so that the defined metamorphic relations should hold.

Metamorphic Relation. The defined transformation of the input should have a known/measurable effect on the output. Checking that this relation holds after the transformation is the *test oracle* of metamorphic testing.

Seed Inputs. The inputs that are used as initial inputs for the tests, to be transformed. If you know the output of the seed input, you may use it to define a stricter relation (output should be correct). But even without the seed output, you can still define a relation check, but it might be a bit more relaxed (output should be similar but you don't know if it is correct).

Metamorphic testing has been defined in 1998 for traditional software systems. While this form of testing was mainly a research theme with few practical applications, this seems to be quite useful when testing machine learning systems.

Let's contextualize metamorphic testing in a key current problem of software engineering for artificial intelligence, i.e., the testing of self-driving cars.

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: Testing

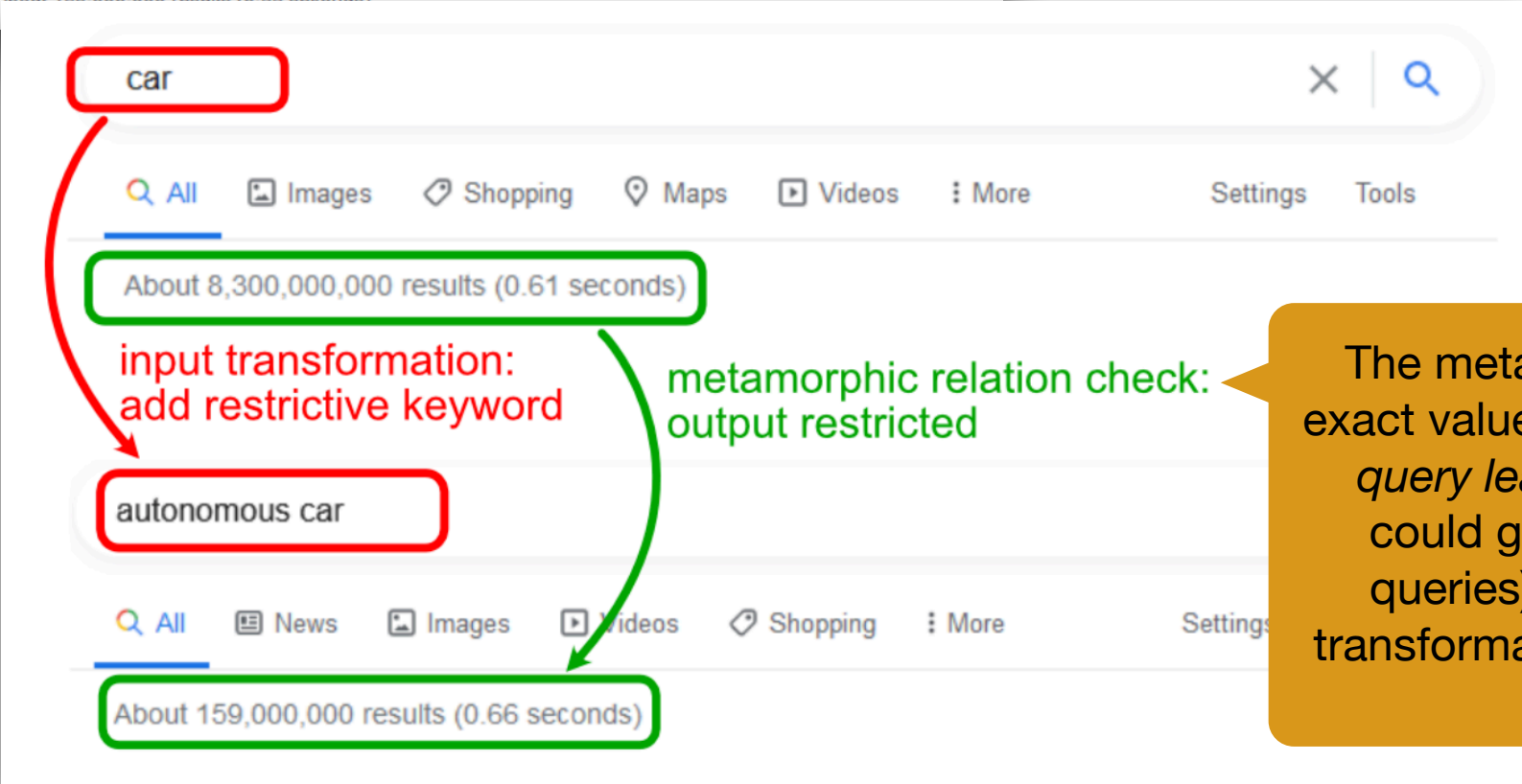
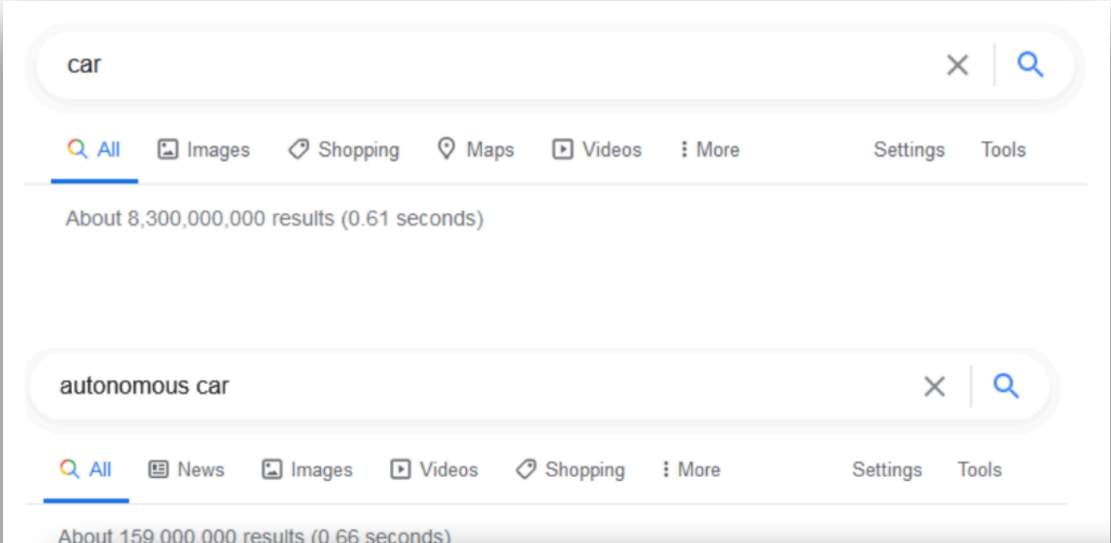
Q5. What is metamorphic testing and how can it be applied for the problem of input testing?

Input testing

Input level tests analyze the training data used to train the ML components as well as the input data used at prediction time.

For each input transformation, metamorphic testing defines a matching relation that should hold when the input transformation is applied.

In the example, we expect to find less results when executing the more restrictive query. This expectation is the relation that we can specify.



The metamorphic test would not specify these exact values, but rather the relation “*restricting the query leads to fewer search results*”. And one could generate several (seed) inputs (search queries), associated restrictive keywords for transformations, and run the query and check the metamorphic relation holds.

Software Engineering for Artificial Intelligence: Ethics and Fairness

Ethical Artificial Intelligence: Achieving Fairness Criteria

Q6. Describe how fairness criteria can be achieved throughout the software lifecycle.

Data acquisition

Testing starts from the beginning! Once you have acquired data, start testing for discrimination. Collect data if performance are poor on some groups.

This is, again, mainly something to tackle at requirements elicitation and analysis time. The analyses of the possible harms, other than the legacy and policy requirements, are indeed done at this time.

Take into account the guideline adopted by US Federal agencies (Department of Justice, Equal Employment Opportunity Commission, etc.) in 1978: the selection rate for a protected group is higher than 80% of the highest rate —> otherwise, the selection procedure is considered as “adverse impact”.

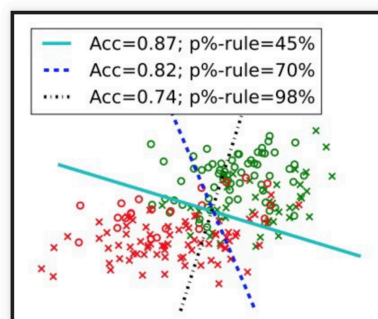
If this is violated, the choice must be justified by business necessities.

Pre-processing

Clean the dataset to reduce correlation between the feature set and the sensitive attributes; in this way, you reduce the influence of sensitive data!

Training time constraints

When training your machine learner, impose additional parity constraints into the ML optimization process, e.g., as part of the loss function.



Remember that artificial intelligence solutions, like traditional software systems, work by means of trade-offs! Explicitly consider fairness as one of the objectives to reach.

Fairness constraints will often lower the model accuracy for some group. That might and might be not fine, depending on the context. Yet, a reasoning of this type is needed!

Post-processing

Adjust the learned model to be uncorrelated with sensitive attributes, for instance by testing multiple ways to set hyper-parameters.

Software Engineering for Artificial Intelligence: Ethics and Fairness

Ethical Artificial Intelligence: Achieving Fairness Criteria

Q6. Describe how fairness criteria can be achieved throughout the software lifecycle.

Data Source

- **Functional:** biases due to platform affordances and algorithms
- **Normative:** biases due to community norms
- **External:** biases due to phenomena outside social platforms
- **Non-individuals:** e.g., organizations, automated agents

Data Collection

- **Acquisition:** biases due to, e.g., API limits
- **Querying:** biases due to, e.g., query formulation
- **Filtering:** biases due to removal of data "deemed" irrelevant

Data Processing

- **Cleaning:** biases due to, e.g., default values
- **Enrichment:** biases from manual or automated annotations
- **Aggregation:** e.g., grouping, organizing, or structuring data

Data Analysis

- **Qualitative Analyses:** lack generalizability, interpret. biases
- **Descriptive Statistics:** confounding bias, obfuscated measurements
- **Prediction & Inferences:** data representation, perform. variations
- **Observational studies:** peer effects, select. bias, ignorability

Evaluation

- **Metrics:** e.g., reliability, lack of domain insights
- **Interpretation:** e.g., contextual validity, generalizability
- **Disclaimers:** e.g., lack of negative results and reproducibility

This because bias can be introduced at any stage of the data pipeline.

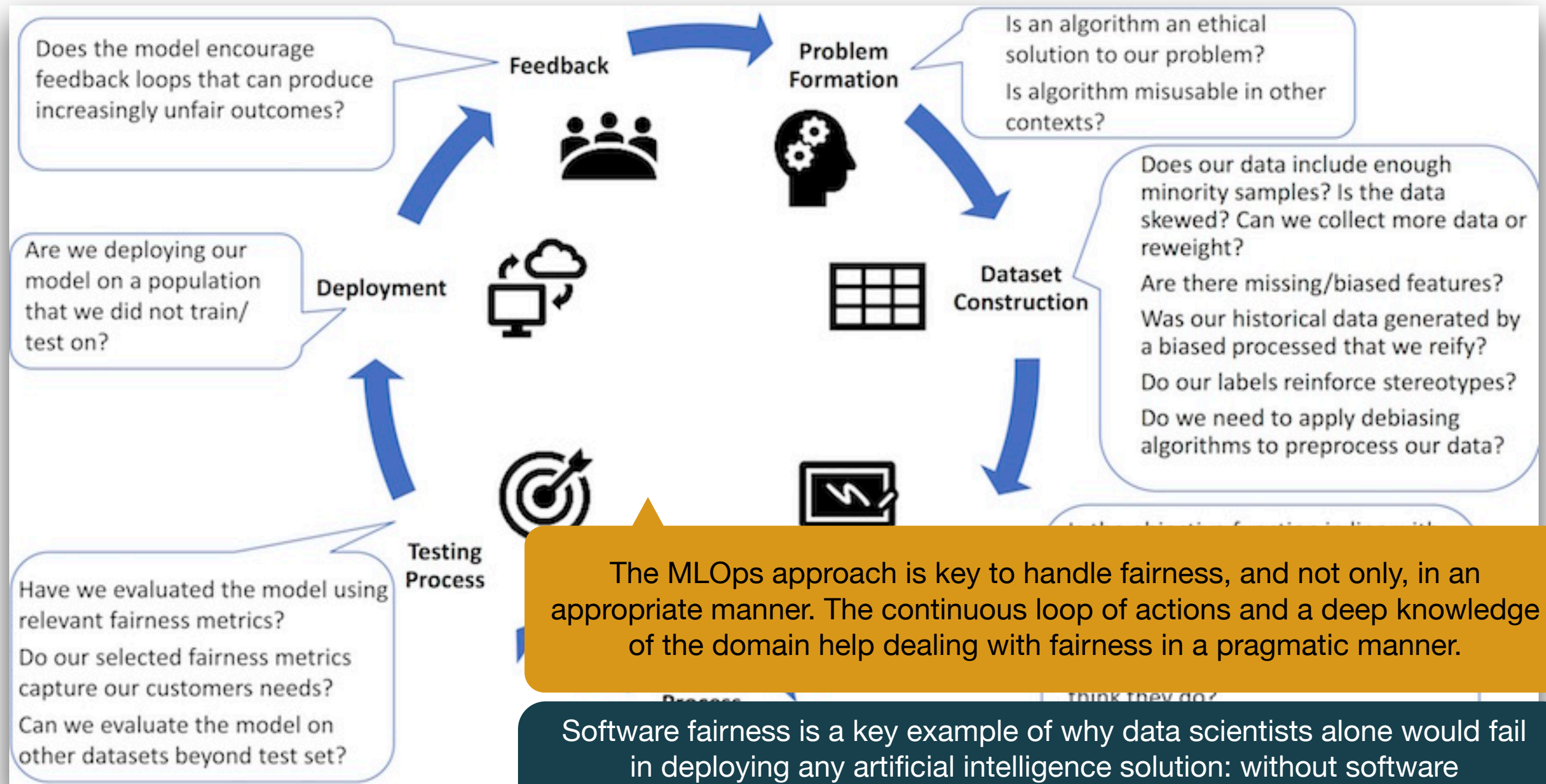
So, data acquisition becomes particularly relevant. You should answer the following questions:

- Address population bias: *Does the dataset reflect the demographics in the target population?*
- Address under- & over-representation issues: *Ensure sufficient amount of data for all groups to avoid being treated as "outliers" by ML, but also avoid over-representation of certain groups (e.g., remove historical data).*
- Data augmentation: Synthesize data for minority groups.
- Fairness-aware active learning: Apply procedures to collect more data for groups with highest error rates.

Software Engineering for Artificial Intelligence: Ethics and Fairness

Ethical Artificial Intelligence: Achieving Fairness Criteria

Q6. Describe how fairness criteria can be achieved throughout the software lifecycle.



The MLOps approach is key to handle fairness, and not only, in an appropriate manner. The continuous loop of actions and a deep knowledge of the domain help dealing with fairness in a pragmatic manner.

Software fairness is a key example of why data scientists alone would fail in deploying any artificial intelligence solution: without software engineering, we would not have any artificial intelligence solutions readily available. Most of them would be enforced to be removed from the market!

Software Engineering for Artificial Intelligence: Security and Privacy

Security and Privacy of Artificial Intelligence Solutions: Attacks on AI Systems

Q7. Provide a classification of the attacks to artificial intelligence systems, explaining how they can impact on security characteristics.

We can classify attacks based on their aim:

Causative attacks

Also known as *poisoning* attacks, aim at influencing the *training* data so that a machine learner does something it is not supposed to do.

Exploratory attacks

Also known as *evasion* attacks, aim at influencing the *input* data so that a machine learner is fed with malicious data since its inception.

Telemetry attacks

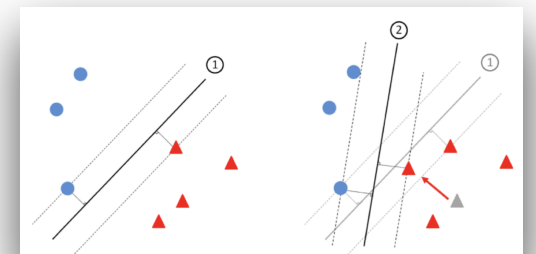
They aim at influencing the *data collection* procedures so that the systems used to collect data provide wrong inputs and, therefore, invalid training data.

These attacks can influence any of the CIA characteristics, i.e., they can affect confidentiality, integrity, and availability. Let's see some examples.

Poisoning attacks and availability

Inject mislabeled training data to damage the model quality. This attack does not necessarily influence sensitive data, but may threat services —> that's why it has to do with availability. It has been shown that 3% poisoning may lead to a decrease of accuracy of around 11%.

This happens because the model's boundary shifts in some way, hence varying the classification capabilities.



Software Engineering for Artificial Intelligence: Security and Privacy

Security and Privacy of Artificial Intelligence Solutions: Attacks on AI Systems

Q7. Provide a classification of the attacks to artificial intelligence systems, explaining how they can impact on security characteristics.

We can classify attacks based on their aim:

Causative attacks

Also known as *poisoning* attacks, aim at influencing the *training* data so that a machine learner does something it is not supposed to do.

Exploratory attacks

Also known as *evasion* attacks, aim at influencing the *input* data so that a machine learner is fed with malicious data since its inception.

Telemetry attacks

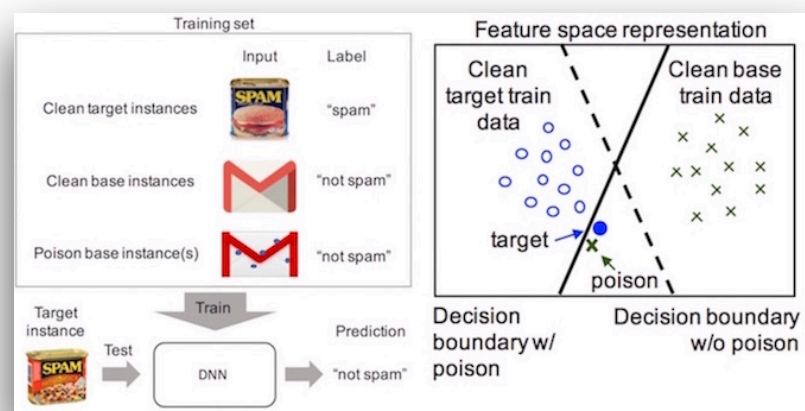
They aim at influencing the *data collection* procedures so that the systems used to collect data provide wrong inputs and, therefore, invalid training data.

These attacks can influence any of the CIA characteristics, i.e., they can affect confidentiality, integrity, and availability. Let's see some examples.

Poisoning attacks and availability

Poisoning attacks and integrity

Insert training data with seemingly correct labels. It may cause misclassification from one specific class to another.



It makes sensitive the decision boundaries, so that a learner cannot classify well anymore a specific class.

—> It is more localized and therefore, more diffused.

Software Engineering for Artificial Intelligence: Security and Privacy

Security and Privacy of Artificial Intelligence Solutions: Adversarial Machine Learning

Q8. Describe the concept of adversarial learning through the use of Generative Adversarial Networks.

Adversarial Machine Learning. Adversarial machine learning is a machine learning method that aims to trick machine learning models by providing deceptive inputs.

Going beyond the schemes: Adversarial Training

All the methods mentioned so far make use of some form of optimization algorithms, attempting to modify inputs or training data according to a certain fitness function.

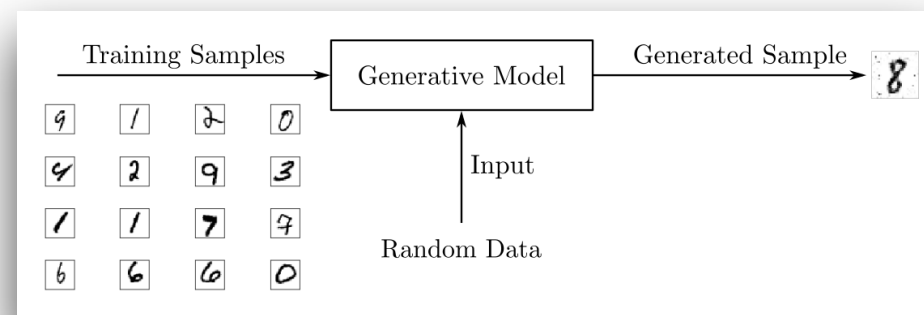
More advanced methods instead rely on learning algorithms. Some of you might have heard already of the so-called *Generative Adversarial Networks* (GAN).

The problem is formulated as a *game with two competing players*—hence, GANs are inspired to game theory.

The players are two neural networks. They play a zero-sum game—one player's gain is equivalent to the other player's loss—where the *generator* tries to produce samples that the *discriminator* will misclassify. Meanwhile, the *discriminator* tries to distinguish real samples from those created by the *generator*.

This means that the discriminator is “*indirectly*” trained: the weights of the neural network will be set such that it is able to discriminate benign and malicious data. Hence, the resulting network will be more robust!

At the same time, the weights of the generator are updated as well so that it may keep improving the way it produces adversarial samples. Hence, the resulting network will provide more and more realistic samples!



We want to generate new fake samples. GANs usually consider a stochastic, or random, element that influences the samples generated by the model.

Software Engineering for Artificial Intelligence: Interpretability and Explainability

Interpretability and Explainability of Artificial Intelligence Solutions

Q9. Explain the difference between interpretability and explainability, also reporting on examples of interpretable and explainable models.

Interpretable models

Models that are intrinsically interpretable, i.e., models simple enough to understand (e.g., short decision trees, sparse linear models).

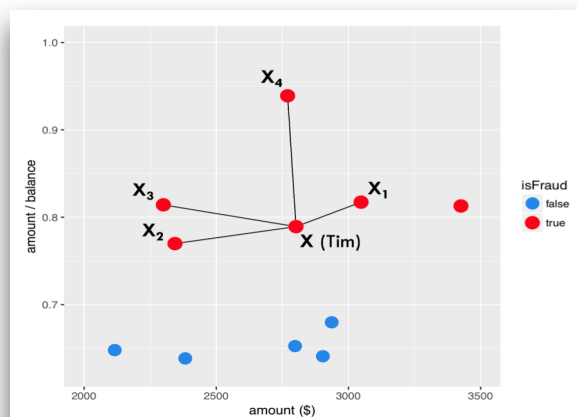
Explainability

It refers to *post-hoc* explanations, i.e., local or global explanations made after that a black-box model makes predictions.

Explanation

This is a justification of a single prediction, typically made in terms of natural language or charts.

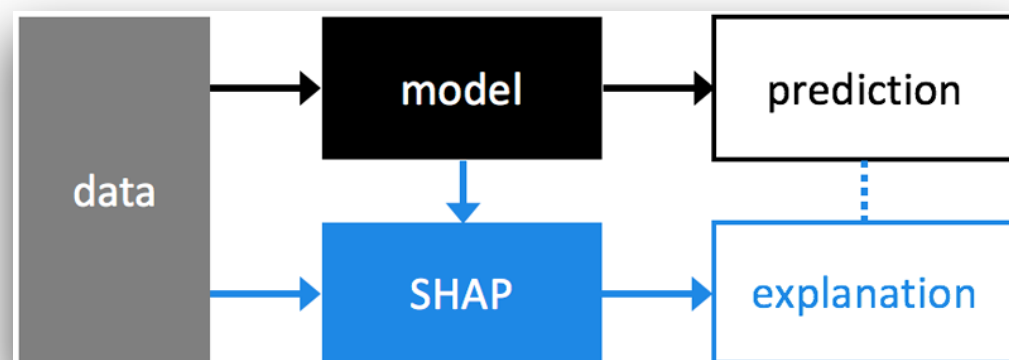
K-Nearest Neighbor



We already discussed this classifier when talking about SMOTE. This is an instance-based learning, that returns the most common class among the k nearest training data points.

It can be considered as inherently interpretable since one can interpret results by showing nearest neighbors. Of course, the interpretation assumes an understandable distance function and interpretable reference data points.

Example: *predict & explain car prices by showing similar sales.*



Explainability takes data and model as input in order to elaborate explanations for predictions on the basis of a local/global surrogate.

Software Engineering for Artificial Intelligence: Interpretability and Explainability

Interpretability and Explainability of Artificial Intelligence Solutions

The general procedure to create global surrogates is the following:

Select a dataset X

The first step consists of using the same dataset used for training the black box model or a new dataset from the same distribution. You could even select a subset of the data or a grid of points, depending on your application.

This is already quite important! For complex tasks, your dataset might be hard to treat (e.g., images). Hence, the ability to select a suitable subset impacts the ability to explain the predictions of the black-box model.

Get predictions

For the selected dataset X, get the predictions of the black box model.

Interpretability step

Select an interpretable model type (linear model, decision tree, etc.).

Training step

Train the interpretable model on the dataset X and its predictions.

Done. You have a surrogate model. Nonetheless, you might want to measure how well the surrogate model replicates the predictions of the black box model. A metric typically used is the R^2 , i.e., percentage of variance explained by the surrogate model. If R^2 is close to 1, then the surrogate model is a good approximation of the black box model.

Someone might ask: *Why should I have a surrogate model if the black box model is able to replicate it well?*

1. Do not forget that surrogate models can be built starting from a subset of the training data!
2. There might be other reasons to use complex models (e.g., non-functional requirements).

But, in any case, if R^2 is close to 1, replacing the complex model with the surrogate is definitively an option!

Software Engineering for Artificial Intelligence

Endterm In-Vivo Quiz

